

Universidad de Antioquia

Facultad de Ingeniería

Informática II — Semestre 2026-1

**Simulación de torneo UdeAWorldCup
en C++ utilizando POO**

Autores:

Gustavo Valencia Mercado
Pablo Gonzalez Rojas

Profesor:

Aníbal Guerra / Augusto Salazar

Medellín, 26 de abril de 2026

Índice

1. Introducción	2
2. Análisis del problema y consideraciones para la alternativa de solución propuesta	2
2.1. Planteamiento del problema	2
2.2. Retos principales	2
2.3. Consideraciones de diseño	3
3. Diagrama de clases de la solución planteada	3
4. Algoritmos esenciales debidamente intra-documentados	5
4.1. Carga de datos desde CSV	5
4.2. Creación de jugadores artificiales	5
4.3. Creación de bombos	5
4.4. Asignación de equipos a grupos con restricciones	6
4.5. Validación por confederación	6
4.6. Programación de partidos de fase de grupos	6
4.7. Simulación de un partido	6
4.8. Ordenamiento de posiciones en grupos	7
4.9. Clasificación a dieciseisavos	7
4.10. Construcción del cuadro eliminatorio	7
4.11. Simulación de rondas finales y estadísticas	8
5. Problemas de desarrollo que se afrontaron	8
6. Evolución de la solución y consideraciones a tener en cuenta en la implementación	9
6.1. Evolución de la solución	9
6.2. Consideraciones importantes de implementación	9
6.3. Posibles mejoras	9
7. Conclusiones	10

1. Introducción

El proyecto *UdeAWorldCup* modela y simula el desarrollo de una copa mundial de fútbol con 48 selecciones. A partir de un archivo CSV con información histórica de los equipos, el sistema construye bombos, conforma grupos respetando restricciones por confederación, programa los partidos, simula la fase de grupos, clasifica los equipos a rondas eliminatorias y finalmente genera estadísticas consolidadas del torneo. La solución fue desarrollada en C++ siguiendo un enfoque orientado a objetos y utilizando estructuras dinámicas.

El objetivo de este informe es documentar el desarrollo de la solución propuesta, justificar las decisiones de diseño adoptadas y describir los algoritmos principales empleados durante la implementación. El contenido se redacta con base en la estructura real del proyecto y en las clases implementadas en el código fuente.

2. Análisis del problema y consideraciones para la alternativa de solución propuesta

2.1. Planteamiento del problema

El problema consiste en representar computacionalmente la organización de un mundial con 48 selecciones, con reglas similares a las de una competición oficial, lo cual implica:

- Cargar y administrar información histórica de cada selección.
- Representar jugadores, partidos, grupos y el torneo completo.
- Formar 12 grupos de 4 equipos a partir de bombos.
- Respetar restricciones de confederación durante el sorteo.
- Programar y simular 72 partidos de fase de grupos.
- Seleccionar clasificados a eliminación directa.
- Construir un cuadro válido para dieciseisavos evitando cruces indebidos entre equipos del mismo grupo.
- Simular las rondas hasta obtener campeón, subcampeón, tercer y cuarto lugar.
- Consolidar estadísticas deportivas y de recursos computacionales.

2.2. Retos principales

El proyecto no solo requería almacenar datos, sino también garantizar coherencia entre reglas deportivas, estado interno del torneo y actualización de estadísticas. Los principales retos fueron:

- Diseñar una estructura de clases suficientemente modular para separar responsabilidades.
- Mantener consistencia entre estadísticas históricas de equipos y estadísticas individuales de jugadores.
- Implementar una estructura dinámica propia para almacenar colecciones sin depender de STL.
- Resolver restricciones en la asignación de grupos y en la configuración de los dieciseisavos.

2.3. Consideraciones de diseño

La alternativa elegida fue una solución orientada a objetos con una clase central llamada *Torneo*, encargada de coordinar el flujo general. Esta decisión permitió encapsular el comportamiento de cada una de las otras clases:

- *Equipo* representa cada selección y sus estadísticas acumuladas.
- *Jugador* representa cada futbolista con sus indicadores individuales.
- *Grupo* encapsula la fase de grupos, su tabla y sus partidos.
- *Partido* modela la simulación de un encuentro concreto.
- *Fecha* abstrae el calendario de programación de partidos.
- *RegistroJugadorPartido* permite separar el rendimiento temporal de un jugador en un partido de sus estadísticas históricas.
- *Recursos* registra iteraciones y memoria estimada.
- *Lista<T>* ofrece una estructura genérica dinámica basada en arreglos redimensionables.

También se optó por una solución secuencial: primero cargar equipos, luego crear jugadores artificiales, después formar bombos y grupos, posteriormente programar y simular los partidos, y al final calcular estadísticas.

3. Diagrama de clases de la solución planteada

La solución implementada se compone de una clase orquestadora principal y varias clases de dominio que colaboran entre sí. La relación estructural es la siguiente:

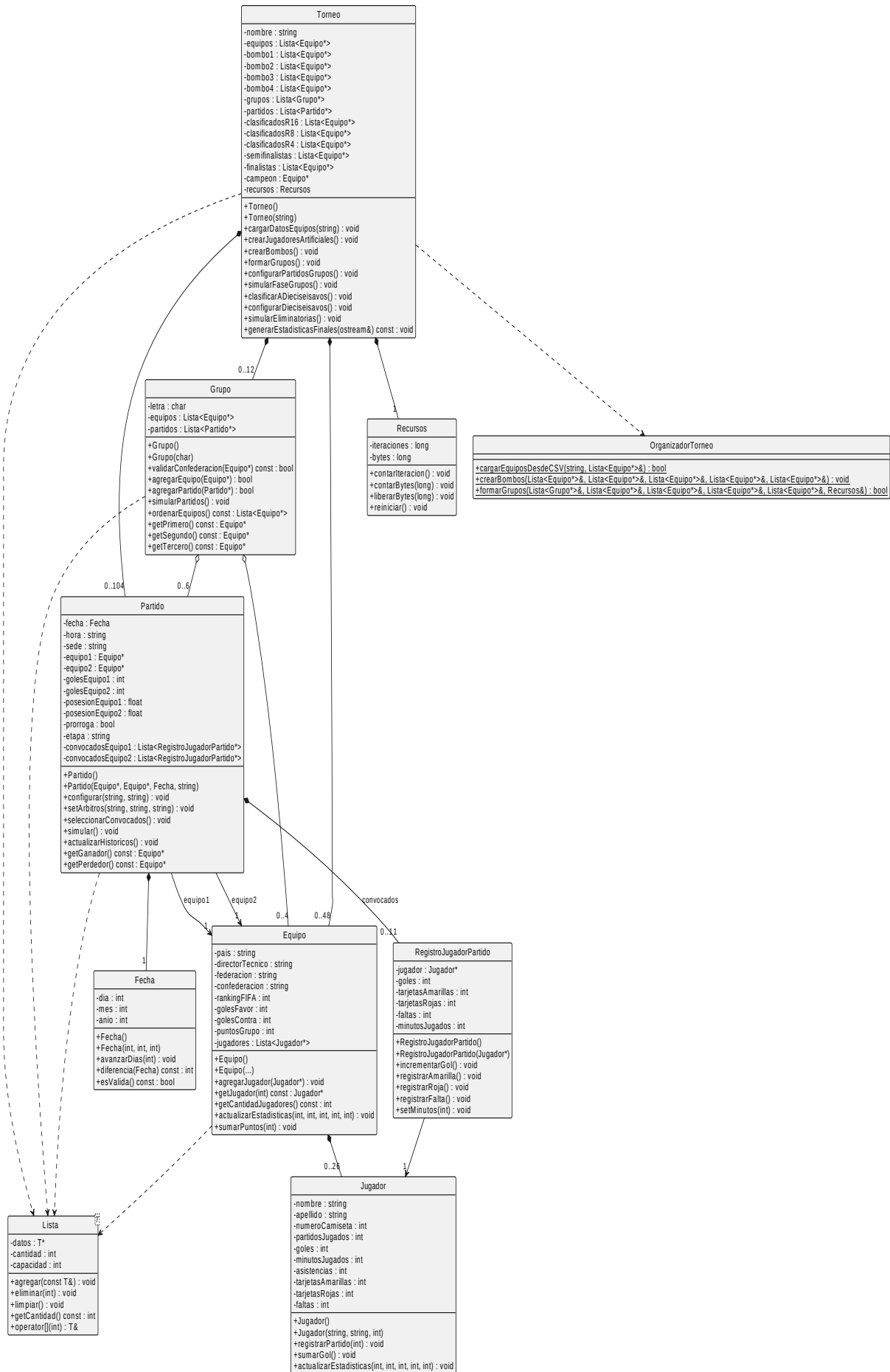


Figura 1: Diagrama de clases resumido de la solución UdeAWorldCup.

- **Torneo:** administra el ciclo completo de la simulación.
- **Equipo:** conserva identidad del país, confederación, ranking y estadísticas globales.
- **Jugador:** almacena desempeño individual acumulado.
- **Grupo:** valida restricciones de confederación, simula partidos y ordena posiciones.
- **Partido:** calcula goles, posesión, convocados, tarjetas, faltas y ganador.
- **Fecha:** permite programar cronológicamente los encuentros.
- **RegistroJugadorPartido:** registra eventos puntuales de un jugador en un partido.
- **Recursos:** registra iteraciones y memoria estimada.
- **Lista<T>:** soporta almacenamiento dinámico reutilizable en todo el proyecto.

4. Algoritmos esenciales debidamente intra-documentados

En este apartado se describen a alto nivel los subprogramas más relevantes de la solución.

4.1. Carga de datos desde CSV

El torneo inicia leyendo un archivo CSV con la información histórica de las selecciones. El algoritmo identifica si el separador es coma o punto y coma, omite la cabecera, separa cada campo, transforma los valores numéricos y crea objetos **Equipo**. Cada equipo se agrega a la colección principal del torneo. Con esto se construye la base de datos de entrada sobre la que opera toda la simulación.

4.2. Creación de jugadores artificiales

Como la entrada del problema no trae la nómina detallada de cada selección, se genera artificialmente una plantilla de 26 jugadores por equipo. Cada jugador recibe nombre, apellido y número de camiseta. Luego, los goles históricos del equipo se distribuyen entre sus jugadores.

4.3. Creación de bombos

Para formar el torneo, los equipos se ordenan por ranking FIFA. El anfitrión se ubica en el primer bombo. Después, los equipos se reparten secuencialmente entre cuatro bombos de 12 selecciones. Esta etapa prepara el sorteo posterior de grupos con una distribución basada en nivel competitivo.

4.4. Asignación de equipos a grupos con restricciones

Uno de los algoritmos más importantes del proyecto es la conformación de grupos. La solución implementa una estrategia de *backtracking*. La idea general es:

1. Crear 12 grupos vacíos, identificados de la A a la L.
2. Determinar para cada posición del sorteo qué bombo corresponde y a qué grupo se debe intentar asignar un equipo.
3. Mezclar aleatoriamente los candidatos del bombo correspondiente.
4. Probar uno por uno los candidatos válidos según la restricción de confederación.
5. Si una asignación lleva a un estado sin salida, deshacer la decisión y probar otra opción.
6. Finalizar cuando todos los grupos tienen 4 equipos válidamente ubicados.

Esta estrategia fue necesaria porque una selección aparentemente válida en un grupo puede bloquear decisiones futuras, por eso no bastaba con una asignación únicamente aleatoria.

4.5. Validación por confederación

La clase `Grupo` implementa la regla de que solo puede haber un equipo por confederación, excepto UEFA, que permite hasta dos. Cada vez que se intenta agregar un equipo, se cuenta cuántos integrantes del grupo ya pertenecen a esa confederación y se decide si la operación es válida. Esta validación es reutilizada tanto durante el sorteo como en la comprobación final de grupos completos.

4.6. Programación de partidos de fase de grupos

Una vez formados los grupos, se generan los seis partidos de cada grupo mediante una matriz fija de combinaciones entre posiciones. El algoritmo distribuye los encuentros en tres rondas y asigna fechas empezando el 20 de junio de 2026. Como se programan cuatro partidos por día, la fecha avanza cada cuatro encuentros usando la clase `Fecha`. Con esto se construye un calendario simple pero coherente para los 72 partidos de la fase de grupos.

4.7. Simulación de un partido

La clase `Partido` concentra la lógica de simulación. El procedimiento sigue varias etapas:

1. Seleccionar convocados para cada equipo, tomando hasta 11 jugadores de la plantilla.
2. Calcular la posesión según una fuerza con base en el ranking FIFA.

3. Estimar los goles esperados de cada equipo a partir del promedio histórico de goles a favor del atacante y goles en contra del rival.
4. Asignar tarjetas, faltas y goles a los jugadores convocados.
5. Si el partido pertenece a eliminación directa y termina empatado, resolver el empate en prórroga con un sesgo influenciado por el ranking.
6. Actualizar las estadísticas históricas de equipos y jugadores.

Este algoritmo no busca reproducir un modelo estadístico profesional, pero sí equilibrar información histórica, ranking y aleatoriedad para producir resultados creíbles.

4.8. Ordenamiento de posiciones en grupos

Para construir la tabla de cada grupo, los equipos se ordenan según:

1. puntos obtenidos;
2. diferencia de gol;
3. goles a favor;
4. sorteo aleatorio final si el empate persiste.

El ordenamiento se aplica sobre una copia de la lista del grupo para no alterar la colección base. Así se obtienen el primero, segundo y tercer lugar.

4.9. Clasificación a dieciseisavos

Al finalizar la fase de grupos, el sistema clasifica:

- los 12 primeros de grupo;
- los 12 segundos;
- los 8 mejores terceros.

Los terceros se ordenan de forma descendente por desempeño en su grupo, mientras que los segundos se ordenan de manera ascendente para favorecer la construcción del cuadro. De esta forma se completa una lista de 32 clasificados.

4.10. Construcción del cuadro eliminatorio

El cuadro de dieciseisavos también se resuelve mediante un algoritmo recursivo. Se toma un equipo y se intenta emparejar con otro clasificado que no provenga de su mismo grupo. Si una combinación futura falla, se retrocede y se prueba otra pareja. Este procedimiento garantiza que el cuadro final no viole la restricción implementada.

4.11. Simulación de rondas finales y estadísticas

Las rondas de dieciseisavos, octavos, cuartos y semifinales se simulan reutilizando un mismo procedimiento general. En cada ronda se crean partidos por parejas, se simulan y se construye la lista de ganadores. En semifinales también se conserva la lista de perdedores para programar el partido por el tercer puesto. Finalmente se obtiene el campeón y se generan estadísticas como:

- ranking final del torneo;
- máximo goleador del equipo campeón;
- tres mayores goleadores del mundial;
- equipo con más goles históricos actualizados;
- confederación con mayor presencia en las rondas finales.

5. Problemas de desarrollo que se afrontaron

Durante el desarrollo de la solución aparecieron varios problemas técnicos y de modelado:

- **Gestión manual de memoria.** El proyecto usa punteros dinámicos para equipos, jugadores, grupos, partidos y registros de partido. Esto obliga a definir con cuidado los destructores y la liberación centralizada para evitar fugas o dobles liberaciones.
- **Diseño de una lista genérica propia.** Al no usar contenedores estándar, fue necesario implementar crecimiento dinámico, copia, eliminación por posición y acceso indexado.
- **Restricciones de confederación.** Un reparto aleatorio simple podía dejar grupos imposibles de completar, por lo que fue necesario usar una solución con retroceso.
- **Consistencia entre niveles de estadísticas.** Los resultados de cada partido debían impactar simultáneamente a equipos y jugadores sin duplicar ni perder información.
- **Empates y desempates.** Fue necesario definir criterios de desempate para tablas de grupo y una forma de resolver empates en llaves eliminatorias.
- **Representación del calendario.** La programación de partidos exigía una clase de fecha que permitiera avanzar días y comparar cronología.
- **Coherencia del cuadro final.** No bastaba con clasificar 32 equipos; también había que asegurar que el emparejamiento de dieciseisavos fuera válido según la regla impuesta.

6. Evolución de la solución y consideraciones a tener en cuenta en la implementación

6.1. Evolución de la solución

La solución puede entenderse como una construcción por capas:

1. Primero se definieron las entidades básicas del dominio: equipos, jugadores, partidos y grupos.
2. Después se creó una estructura de almacenamiento genérica para soportar el resto del sistema.
3. Luego se implementó la clase **Torneo** como coordinadora del flujo completo.
4. Más adelante se añadieron las reglas del sorteo, la programación de partidos y la clasificación.
5. Finalmente se incorporaron detalles complementarios como la clase **Fecha**, el registro individual por partido y la medición de recursos.

Esta evolución muestra que el proyecto no se resolvió como un único bloque, sino como una integración progresiva de componentes con responsabilidades bien delimitadas.

6.2. Consideraciones importantes de implementación

Al implementar una solución de este tipo conviene tener en cuenta lo siguiente:

- El orden de ejecución de las etapas es fundamental. No se pueden formar grupos sin bombos, ni simular partidos sin haberlos configurado previamente.
- Cuando se trabaja con punteros compartidos entre varias clases, debe quedar claro quién es responsable de liberar cada objeto.
- Las funciones que resuelven restricciones combinatorias deben diseñarse pensando en estados parciales inválidos y en la posibilidad de retroceder.
- Las estadísticas deben actualizarse una sola vez por partido para evitar inconsistencias.
- La implementación de estructuras propias obliga a prestar especial atención a copias, asignaciones y re-dimensionamientos.

6.3. Posibles mejoras

Aunque la solución cumple con los objetivos planteados, existen mejoras posibles:

- incorporar lectura real de plantillas de jugadores desde archivo;

- usar probabilidades más refinadas para la simulación de partidos;
- programar fechas distintas para cada ronda eliminatoria;
- ampliar los criterios de desempate con enfrentamiento directo;
- separar aún más la lógica de simulación, entrada/salida y estadísticas.

7. Conclusiones

El proyecto *UdeAWorldCup* resuelve de forma consistente la simulación de un mundial de 48 selecciones mediante una arquitectura orientada a objetos y una implementación propia de estructuras dinámicas. La solución no se limita a imprimir resultados, sino que modela entidades, reglas de negocio, restricciones de clasificación, programación de partidos y estadísticas finales.

Desde el punto de vista académico, el trabajo evidencia aplicación de abstracción, encapsulamiento, composición, manejo de memoria dinámica, recursión y diseño modular. Además, muestra cómo una descomposición adecuada del problema permite integrar progresivamente lectura de datos, simulación, control de restricciones y generación de reportes en una sola aplicación coherente.

En conjunto, la implementación satisface los requerimientos funcionales planteados y constituye una base válida para futuras extensiones, ya sea a nivel de realismo estadístico, presentación de resultados o ampliación de reglas del torneo.